

専門コース演習 III Python CGI入門

担当: 石橋 直樹
E-Mail: n-ishi@musashino-u.ac.jp

1 講義の目的

Web サービスにおいて対話的 (インタラクティブ) な情報の配信を行うために、Python を用いた CGI(Common Gateway Interface) プログラミングの技法を学ぶ。

2 CGI とは...

CGI とは、何らかのプログラミング言語で書かれたプログラム (コンピュータに依頼する命令を記述したもの) で、1) ユーザから問い合わせを受け取り、2) サーバ上でプログラムを実行し、3) 結果を HTML で出力する。

これまで学んだ HTML の記述では、表示したい HP の書式や内容を記述したが、CGI では、出力に求められる様々な演算 (命令) を実行することで、HTML の形式で記述されたデータを自動的に生成する。

3 Hello World 1-sample01.py

以下のソースコードは、アクセスすると単純に "Hello, world!" と出力する。ここで、その出力が HTML の形式で出力されることがわかる。

```
1 #!/usr/bin/python
2 import cgi
3 import cgitb
4 cgitb.enable()
5
6 print("Content-Type: text/html")
7 print()
8
9 print('<html><head></head>')
10 print('<body>')
11 print('<p>Hello, world.</p>')
12 print('</body></html>')
```

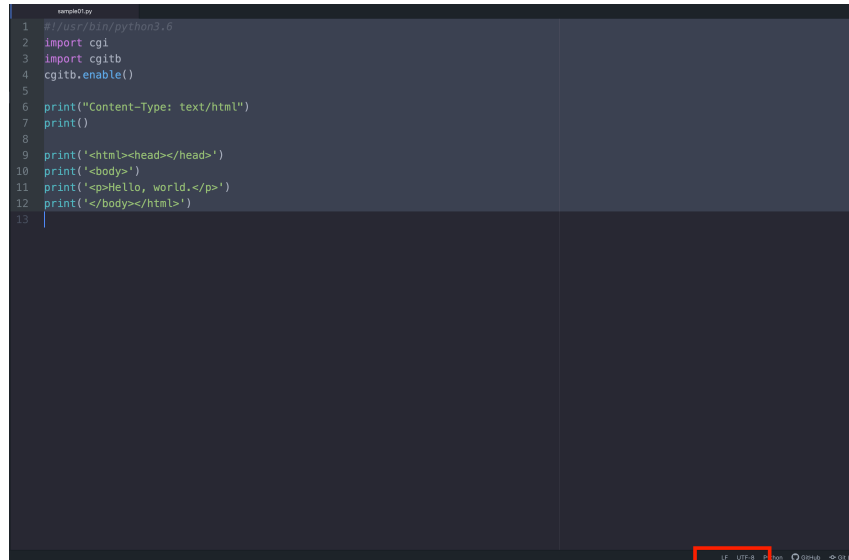
4 ファイルを作成する

本講義では、ATOM を用いて、CGI プログラムの作成を行う。各自、まず、同ソフトを起動して、上記のプログラムを記述すること。

なお、実習でもちいるサーバを使用する上で、保存する際、次の点に注意すること。

- 名前は半角英数字に、".py" をつけたものとする (eg. sample01.py)。
- 文字コードは、utf-8 を用いる。
- 改行コードは、LF を用いる。

1. ATOM の画面において、前述の CGI プログラムを記入する。
2. 文字コードを”utf-8”、改行コードを”lf”に設定する。



```
1 #!/usr/bin/python3.6
2 import cgi
3 import cgitb
4 cgitb.enable()
5
6 print("Content-Type: text/html")
7 print()
8
9 print('<html><head></head>')
10 print('<body>')
11 print('<p>Hello, world.</p>')
12 print('</body></html>')
13
```

3. 出来上がったら、[File] メニューから、[Save As...] を選択する。
4. "sample01.py" など、適当な名前を設定し、本講義用のフォルダを作るなど、適宜場所を決めて、保存する。

5 サーバへのファイル転送方法

[手順 1] Cyberduck の起動

ファイル転送を行うためのソフトウェア、「Cyberduck」を起動する。

[手順 2] サーバの指定

Cyberduck 画面において、実習用サーバ (muds.gdl.jp) を選択し、接続する。

[手順 3] サーバの指定

Cyberduck 画面において、「public.html」フォルダへ移動する。

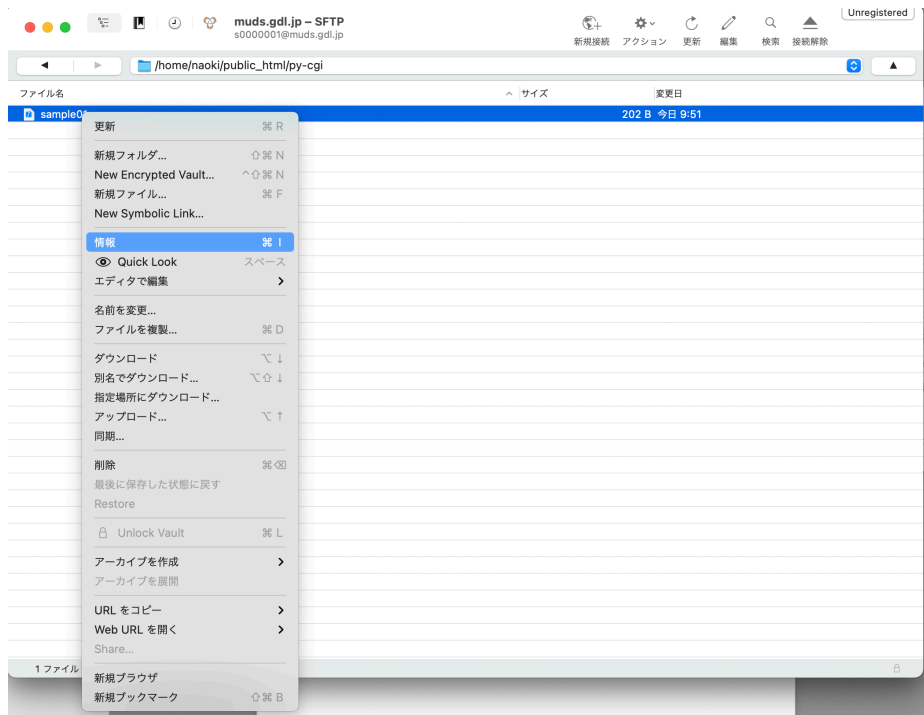
[手順 4] ファイルの転送

転送したいファイル (たとえば sample01.py を Cyberduck の画面へドロップすると、転送が実行される。

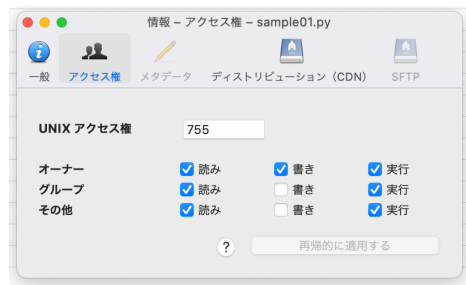
[手順 5] 実行権限の設定

ネットワークを経由して、全てのユーザが、この CGI プログラムを動かす (実行する) 事ができるよう、設定を変更する。

送信した py ファイルを右クリック (Ctrl-Click) し、「情報」を選択する。



「実行」を全てチェックし、「OK」を押す。ただし、「書き」はオーナーだけとすること。結果、「755」と上部に出ていれば正しく設定されている。



[手順 6] 動作確認

以上の作業で生成される CGI ファイルは、次のような URL で実行できる。

URL: <http://muds.gdl.jp/~s0000001/sample01.py>

6 Hello World 2-sample02.py

html の `<form>` タグを用いて送られてきたデータの受け取り方を示す。cgi モジュールを読み込むことで、FieldStorage メソッドが使用でき、これをもちいて送られてきたデータを配列として受け取ることができる。

```

1 #!/usr/bin/python
2 import cgi
3 import cgitb
4 cgitb.enable()
5 form = cgi.FieldStorage()
6 print("Content-Type: text/html")
7 print()
8 print('<html><head></head>')
9 print('<body>')
10 print('<form action="./sample02.py" method="POST">')
11 print('氏: <input type="text" name="ln">')
12 print('名: <input type="text" name="fn">')
13 print('<input type="submit">')
14 print('</form>')
15
16 if "fn" not in form or "ln" not in form:
17     print("<p>氏名を入力してください。</p>")
18 else:
19     fn=form["fn"].value
20     ln=form["ln"].value
21     print('<p>Hello, %s %s</P>' % (fn, ln))
22
23 print('</body></html>')

```

7 PostgreSQL を用いたチャット-sample03.py

PostgreSQL を Python CGI で用いる例として、チャットを構築してみる。まず、PostgreSQL に次の SQL を送ることで、チャットのデータを受け取るリレーションが作られる。

```

1 create table pychat(
2     cid serial primary key,
3     uname text not null,
4     message text not null,
5     pdate timestamp not null
6 ) without oids;

```

PostgreSQL を Python で使うため、この例では psycopg2 モジュールを利用する。同モジュールで PostgreSQL サーバへ繋ぐため、まず connect メソッドで接続を定義する。さらに、cursor メソッドを使ってデータベースへの操作を行い、commit メソッドで SQL を実行する。

```

1 #!/usr/bin/python
2 import cgi
3 import cgitb
4 import psycopg2
5 cgitb.enable()
6 connection = psycopg2.connect("host=localhost dbname=s0000001 user=s0000001 password=
    XXXXXX")
7
8 form = cgi.FieldStorage()
9 print("Content-Type: text/html")
10 print()
11 print('<html><head></head>')
12 print('<body>')
13 print('<form action="./sample03.py" method="POST">')
14 print('ユーザ名: <input type="text" name="uname"><br />')
15 print('message: <input type="text" name="message" size="40"><br />')
16 print('<input type="submit" value="chat!">')
17 print('</form>')
18
19 if "uname" in form and "message" in form:
20     uname=form["uname"].value
21     message=form["message"].value
22     cur = connection.cursor()
23     cur.execute("insert into pychat(uname,message,pdate) \
24         values('%s','%s',current_timestamp);" % (uname, message))
25     connection.commit()
26     cur.close()
27

```

```

28 cur = connection.cursor()
29 cur.execute("select cid,uname,message,pdate from pychat order by cid desc;")
30 print("<ul>")
31 for row in cur:
32     print("<li>%s: %s (%s)</li>" % (row[1],row[2],row[3]))
33 cur.close();
34 print("</ul>")
35
36 print('</body></html>')

```

8 その他モジュールの追加について

本講義で用いるサーバについて、python の他のモジュールが必要な場合、ターミナルで `muds.gdl.jp` へつなぎ、次のコマンドで自分の環境として入れる事ができる。

```

1 pip3 install [モジュール名] --user

```

9 おまけ: SQL 復習

本講義では、PostgreSQL を用いた時空間データベースの構築実習を行う。PostgreSQL はオブジェクトリレーショナルデータベース管理システム (ORDBMS) の一つで、テキスト・数値などの基本的なデータ構造に加え、点、線などの幾何的データを SQL に用いることができる。本講義では、データベースの構築と、時空間的検索について述べる。

9.1 リレーシヨンの作成

リレーシヨンを作成するためには、次のように記述される CREATE TABLE 文を用いる。

```

1 CREATE TABLE リレーシヨン名 (
2   属性名1 データ型1,
3   属性名2 データ型2,
4   :      :
5   属性名n データ型n
6 );

```

属性名 1～n には、属性の名前 (半角英数字のみ使用可能)、データ型 1～n には、以下で示すデータの型を記述する。

型名	説明
int4	4 byte の整数。
int8	8 byte の整数。
float4	4 byte の少数。
float8	8 byte の少数。
text	長さ不定の文字列。

9.2 データの挿入、および、更新

データをリレーションへ挿入するためには、次の SQL 文を用いる。

```
1 INSERT INTO リレーション名 (属性名 1, 属性名 2, ..., 属性名n)
2 VALUES (値 1, 値 2, ..., 値n);
```

なお、データの挿入を行う際、' でデータを囲み、記述する (例: 'abcd')。
また、データの更新を行うためには、次の SQL を用いる。

```
1 UPDATE リレーション名
2 SET 属性名 1=値 1, 属性名 2=値 2, ..., 属性名n=値n
3 WHERE 条件 1
4 AND 条件 2
5 AND ...
6 AND 条件m;
```

9.3 データの削除

リレーションからデータを削除する場合、次の SQL 文を用いる。

```
1 DELETE FROM リレーション名
2 WHERE 条件 1
3 AND 条件 2
4 AND ...
5 AND 条件m;
```

9.4 リレーションの削除

リレーションを削除するためには、次の SQL 文を用いる。

```
1 DROP TABLE リレーション名;
```

9.5 検索

検索は SELECT を用いて行う。すべての属性を検索する場合は*で表現する。

```
1 SELECT 属性名 1, 属性名 2, ..., 属性名n
2 FROM リレーション名
3 WHERE 条件 1
4 AND 条件 2
5 AND ...
6 AND 条件m;
```

10 PostgreSQL の起動

ターミナル (Mac)、TeraTerm(Windows) を用いて、サーバ muds.gdl.jp に接続し、次のコマンドを打ち込み、パスワード認証の後、PostgreSQL を使用することができる。

```
1 [s0000001@muds ~]$ psql -U s0000001 -W s0000001
```

ちなみに”-U s0000001”はユーザ名を指定、”-W”はパスワード認証、末尾の s0000001 はデータベース名を表す。PostgreSQL の ID とパスワード等は、各ユーザのホームディレクトリに readme.txt として設置してある。本サーバでは、ユーザ名と同名のデータベースを各自に設定しているため、自分のデータベースを開く場合、次のように略することができる。

```
1 [s0000001@muds ~]$ psql
```

正常に起動すると、次のような SQL インタプリタが起動される。ここに、SQL を打ち込むことで、自分のデータベースを操作できる。

```
1 ユーザ s0000001 のパスワード:
2 psql (12.7)
3 "help"でヘルプを表示します。
4
5 s0000001=>
```

ちなみに、create 文、insert 文を含むテキストファイルを作成、Cyberduck 等を用いてサーバへ転送し、次のように読み込ませることもできる (ファイル名が file.sql の場合)。

```
1 [s0000001@muds ~]$ psql < ./file.sql
```

SQL インタプリタを終了する場合、次のように入力する。

```
1 s0000001=> \q
```
